

10.4 Example: Automated Commenting of Code

Many programmers prefer to focus on the actual code rather than the tedious task of adding comments to it. But if someone else then has to further develop or maintain the code, good advice is hard to come by. So, we had the following idea: couldn't the AI tool add comments to existing code? Initial tests in ChatGPT have been consistently successful (see also [Chapter 6](#)). We then tried to automate the process and used the following role description:

Role: You are a coding assistant. Your goal is to add missing comments to existing code. At the very least, add a header to the file to indicate its purpose. Try to explain the purpose of each function/method.

If necessary, add comments to explain blocks of code. Don't change/remove existing comments.

Apart from comments, leave the code as it is. The functionality of the code MUST NOT be changed.

DO NOT ADD EXPLANATIONS in your response.

DO NOT USE MARKDOWN to format your response.

When you skim the code in the following code, the basic procedure should be clear straight away.

```
# Sample file openai/comment-files.py
#!/usr/bin/env python3
import os
from openai import OpenAI
client = OpenAI() # reads the .env file

# searches recursively for all files with specified
```

```

identifiers
def find_files_with_extensions(directory, extensions):
    matching_files = []
    for root, _, files in os.walk(directory):
        for file in files:
            if any(file.endswith(ext) for ext in
extensions):

matching_files.append(os.path.join(root, file))
    return matching_files

# reads a code file, adds comments and saves them
def comment(filename):
    # Read file, create backup
    with open(filename, "r") as file:
        content = file.read()
    with open(filename + ".bak", "w") as backup:
        backup.write(content)
    # Put together roll and prompt
    role = ""You are a coding assistant. Your goal
...""
    prompt = f""Please add comments to the
        following code:\n\n{content}""
    try:
        completion = client.chat.completions.create(
            model = "gpt-4o",
            temperature = 0.3,
            messages = [ {"role": "system", "content":
role},
                        {"role": "user", "content":
prompt} ] )
    except Exception as e:
        print(" An error occurred:", str(e))
        return

    # Analyze result, save modified code file
    if completion.choices[0].finish_reason != "stop":
        print(" Commenting failed.")
        print(" Input Tokens:",
completion.usage.prompt_tokens)
        print(" Output Tokens:",
completion.usage.completion_tokens)
    else:

```

```

        new_code =
completion.choices[0].message.content.strip()
        with open(filename, 'w') as file:
            file.write(new_code)
        print("    Commenting done.")
        print("    Input Tokens:",
completion.usage.prompt_tokens)
        print("    Output Tokens:",
completion.usage.completion_tokens)

# Main
dir = "/Users/kofler/my-sample-files"
extensions = [".js", ".php"]
files = find_files_with_extensions(dir, extensions)
for filename in files:
    print(filename)
    comment(filename)

```

The formulation of the role is decisive for the functionality of this script. There you can specify whether the language model should comment more or less, whether a special syntax (e.g., JavaDoc) should be adhered to when commenting, and so on.

Tip

If you want to try out the script with your own files, copy your code into a separate directory and remove the existing comments there. (I'm sure you're one of the developers who comment well, aren't you?)

A useful tool for removing comments is `cloc` (<https://github.com/AlDanial/cloc>). By using `cloc --strip-comments=nc myfile.py`, you create the new `myfile.py.nc` file, which doesn't contain any comments. Instead of `.nc` (*no comments*), you can use any combination of letters.

10.4.1 Real-Life Experience

We tested the commenting script not only with the OpenAI API, but also with Ollama and Groq. Corresponding scripts are included in the sample files for the book. The results we achieved were mixed:

- **OpenAI with GPT-4o**

The automatic commenting feature worked quite well for small code files up to approximately 10 KB. Of course, even a language model can't be clairvoyant, but in many cases the comments were appropriate in terms of content and at least suitable as an initial aid to understanding the code.

Of course, you can argue about the usefulness of some comments. If a self-defined JavaScript function is called `updateTopicsWithChapters`, and the language model adds the comment *Function to update topics with chapters* in the line above it, there isn't much added value. But there are also more positive examples. In our PHP test files, the AI tool has added a description of all parameters, the return value, and any exceptions to each method.

The main problem with the OpenAI API has turned out to be its limitation to 4,096 output tokens. We've looked for various ways to circumvent this limit ([Section 10.4.4](#)), but have failed.

- **OpenAI with GPT-4o-Mini**

Surprisingly, `gpt-4o-mini` was hardly faster than `gpt-4o` in our tests. The quality of the comments was comparable. Fortunately, `gpt-4o-mini` can handle much larger files because the maximum number of output tokens is much higher. However, `gpt-4o-mini` was very sparing in its comments, especially with long files, and limited itself to just one-line descriptions of the functions.

- **Ollama**

On our test computer (MacBook with M3 Pro CPU), Ollama can only execute relatively small LLMs with up to 10 billion parameters at a reasonable speed, for example, `llama3:8b`. We've also experimented with `gemma2`, `codellama`, and `codestral`, but the LLMs tested were unable to cope with the task, especially with larger files. It happened time and again that the answer didn't contain the code with comments, but a relatively short textual description of the code file. That is missing the point.

- **Groq**

Commenting smaller code files with language models hosted on Groq's servers worked perfectly. This is astonishing in that some of the same language models were used as in Ollama, albeit with larger context windows and more parameters (e.g., `llama3-70b-8192` or `mixtral-8x7b-32768`).

Groq also failed with large code files. In this case, it was due to the rate limit of the developer access. The commenting of only *one* large code file already exceeds the limit for tokens/minute and is therefore rejected.

We would have liked to repeat our tests without a rate limit and would have been prepared to pay for a corresponding test account, but our inquiries in this regard weren't answered until the text for this book had already been completed. Groq seems to be aimed more at large enterprise customers and less at small developers.

Fine-Tuning

To achieve better results, you can prepare a set of sample files, one uncommented and one with comments that you yourself consider the best solution. From these sample files, you create prompt-response pairs that you embed in the `messages` array. This provides the language model with some patterns it can use as a guide (also with regard to the desired formatting of the response).

However, this approach didn't deliver convincing results in our tests. Instead, we had the impression that the additional examples in the context window tended to confuse the language model, especially if the content of the examples had nothing to do with the code files to be processed later or were even available in a different programming language. This approach also increases costs because the examples have to be sent with every API call, which greatly increases the number of input tokens.

You can pursue the same idea more effectively if you use your collection of examples to fine-tune the language model. Background information and tips on the procedure for fine-tuning OpenAI language models and local language models (Ollama) can be found on the following pages:

- <https://openai.com/index/gpt-4o-fine-tuning>
- <https://github.com/ollama/ollama/issues/2488>
- <https://huggingface.co/docs/transformers/training>

10.4.2 Extracting Code from the Response

Only GPT-4o and GPT-4o-mini consistently delivered the code without Markdown formatting and without introductory text (e.g., “Here is the code with added comments”) or subsequent explanations, as we had requested. In all the other language models we tested, the answer was repeatedly embellished in different ways. We’ve tried to extract the code from the response using the Python function, `extract_code`. Although this worked reliably in our tests, it’s of course not a good sign if the language model isn’t sufficiently “intelligent” to understand the task.

```
# in the sample file groq/comment-files.py
def extract_code(md):
    md = md.strip()
    lines = md.split("\n")
    first, last = 0, len(lines)
    for i in range(len(lines)):
        if lines[i].startswith("`"):
            first = i + 1
            break

    if first == 0:
        return md
    else:
        print("  Markdown found")
        for i in range(first + 1, len(lines)):
            if lines[i].startswith("`"):
                last = i
                break
        return "\n".join(lines[first:last])
```

10.4.3 Genuine Errors

During our tests, we only came across one serious error: GPT-4o tried to comment the SQL code embedded in a PHP file. This would have been fine if it had introduced the comments with `--` according to the SQL standard. Instead `//` has been used, which is specific to GPT-4o and caused the SQL command to trigger an error.

We've greatly simplified the following listing to illustrate the problem. In fact, it was a fairly complex `SELECT` command of more than 20 lines.

```
// before
$sql = sprintf("
    SELECT n.*, CONCAT(p.firstname, ' ', p.lastname) AS
fullname
    FROM notes n
    JOIN person p ON p.id = n.author
    WHERE n.topic = %u
    ORDER BY n.ts", topic_id);

// after
$sql = sprintf("
    SELECT n.*, CONCAT(p.firstname, ' ', p.lastname) AS
fullname
    FROM notes n
    JOIN person p ON p.id = n.author
    WHERE n.topic = %u
    // sort by timestamp      <-- incorrect comment!
    ORDER BY n.ts", topic_id);
```

Even if this error was an isolated case (usually, SQL code is handled correctly, even if it's embedded in code files with other programming languages), the example proves that you must always check the automatically inserted comments. The best way to do this is to use an editor such as Visual Studio Code (VS Code) and compare the commented file with its status at the last commit.

10.4.4 Trouble with Large Code Files

The main problem with this example is that all APIs are overwhelmed by large code files for various reasons. We've created several variants of the script printed previously in which we tried to circumvent this restriction. (We only conducted these experiments using the OpenAI API, not any other API.)

Unfortunately, all attempts have failed:

- The obvious thing to do is to split the code file into pieces, that is, lines 1 to 100, then lines 101 to 200, and so on. With each call, the language model receives 100 lines and is supposed to add comments to these. But that's not a good idea. First, the context is lost. The language model can no longer analyze the code in its entirety; for example, based on only the first 100 lines, it can't estimate what the overall purpose of the entire file code is. Second, the code is often disassembled in unfavorable places. Nonmatching parenthesis levels, unclosed character strings, and so on make the pieces of code syntactically incorrect. If you wanted to attribute human traits to the language model, you could say that these errors unbalance the model. Contrary to all instructions, it attempts to correct the code.
- It's a little smarter to pass the entire code file to the language model each time. This requires a language model with a sufficiently large context window. The desire for a lot of context is widespread, so more and more new models are touting their ability to deal efficiently with a large context. For example, GPT-4o allows a context size of 128,000 tokens. This is sufficient even for huge files.

In the prompt, you then ask the model to comment and return only lines 1 to 100 the first time it's called, 101 to 200 the next time, and so on. The main difference to the previous strategy is that the language model always has access to the entire code file, even if it only processes and returns a line range each time.

Our attempts failed mainly because GPT-4o didn't adhere exactly to the line numbers. For example, the model was supposed to process lines 1 to 100. In line 90, a method started whose code extended to line 110. The model has considered and returned the entire method. To ensure that the code is syntactically correct, it has added a parenthesis after the method to close the class. However, this parenthesis, which doesn't appear at this point in the original file, is too much if individual pieces of code are to be reassembled later.

- Our next idea was to ask the language model to return only DIFF patches instead of the new, complete code. There are two advantages to this. First, the patches only contain the changes, are therefore smaller, and use fewer tokens. Thus, more code can be processed at once without exceeding the output token limit.

Second, we thought that applying the patches would be more robust than merging the pieces of code from variant 2, but we were wrong: the quality of the patches was poor, and many changes couldn't be correctly applied to the original code file. Some of the errors were trivial; often the line number was simply wrong. We then tried context patches or unified patches (`diff` options `-c` or `-u`). That didn't make things any better. Language models obviously don't think in DIFF patches any more than humans do.

At this point, we gave up our experiments. Ultimately, there is no way around editing each code file in its entirety. The only solution is to wait for language models or APIs that have a large context window and allow a large number of output tokens. It's quite possible that such offers are widespread by the time you read this book.

10.4.5 Translating Existing Comments

A variation on the original task is the translation of comments. Many projects start small. Comments are written in the respective locale (if at all). If the project is surprisingly successful after 10 years and is about to be sold, the German (or French or Italian) comments in the code suddenly no longer make a good impression. They stand in the way of further development of the code by an international team and reduce the value of the software.

Basically, nothing changes in the structure of the script presented earlier. Only the role description and prompt formulation need to be adapted:

```
# Sample file openai/translate-comments.py
...
role = """You are a coding assistant.
    Your goal is to translate non-English comments
into English.
    Apart from comments, leave the code as it is.
    The functionality of the code MUST NOT be changed.
    DO NOT ADD EXPLANATIONS in your response.
    DO NOT USE MARKDOWN to format your response."""
prompt = f"""In the following code, please replace the
German
```

```
comments with English comments:\n\n{content}"""
```

...

Our tests with the OpenAI API (GPT-4o) were absolutely satisfactory, albeit within the limitations already described. The script therefore fails with code files whose size exceeds the context window or the number of output tokens.

In real life, often not only are the comments not in English but also variable and function names, table and column names, and so on. In view of previous experience, we hope that we don't need to explain here that automated anglicization of the code base of your project is miles away from the possibilities of current AI tools.

The main problem is that the change of a method name in the file of a class must also be reproduced in all other files that use this class. It's therefore impossible to treat each file separately. Rather, the code base must be considered in its entirety. Good editors and IDEs are able to do this. It would be great to have an interface between the editor and the AI API to combine the capabilities of both tools. For the time being, this is a dream of the future.

10.4.6 Conclusion

We started work on this section with the expectation of being able to present a simple yet useful example. We were thoroughly mistaken for the following reasons:

- Many LLMs aren't even consistently able to fulfill the task precisely defined in the system description (role) and in the prompt.
- The annotation of smaller files works reasonably well. With large files, commenting is only successful if the context window is large enough to process the entire file and if the response size isn't limited.
- In very rare cases, the language models changed the code in such a way that it no longer worked properly. A thorough check of the changes is absolutely essential.
- From ChatGPT and others, we're used to language models working almost without delay. You'll have to bid farewell to this idea as soon as

you apply the scripts presented here to 10, 100, or possibly even more files. Depending on the API or your own hardware, the execution may take minutes or, in the worst case, hours. This makes the testing and optimization process very laborious.